

MedBill AI

Automated Medical Bill Scrutiny System for PSU Employees

End-to-end pipeline: Scanned PDF -> OCR -> AI -> Validated Claim Report

23

Bugs Found & Fixed

39

Validation Rules

6

Core Source Files

github.com/kumarnitn/MEDICAL-BILL-SCRUTINY-SYSTEM

March 2026 - FastAPI + Tesseract OCR + Ollama Phi-3 + SQLite

Table of Contents

- 1. Project Overview
- 2. System Architecture
- 3. Processing Pipeline (4 Stages)
- 4. File-by-File Guide
- 5. Validation Rules Engine (39 Rules)
- 6. API Reference
- 7. Dashboard Guide
- 8. All 23 Bugs Found & Fixed
- 9. Setup & Running Locally
- 10. Deployment

1. Project Overview

MedBill AI automates scrutiny of medical bills submitted by PSU employees under CPRMSE/CPRMSNE schemes of Coal India Limited (CIL) and subsidiaries. Medical claim officers manually review hundreds of scanned hospital bills -- checking CGHS rates, room entitlement, hospital empanelment, documentation, and fraud. MedBill AI automates extraction and initial validation, flagging issues for officer review.

Key Capabilities

- OCR Extraction -- Converts scanned bill PDFs to text via Tesseract
- AI Structuring -- Uses local LLM (Phi-3 via Ollama) to parse OCR text into structured JSON
- Rule Validation -- 39 deterministic rules check eligibility, rates, docs, fraud
- CGHS Rate Lookup -- Matches each line item against official CGHS rate database (3,000+ procedures)
- Hospital Empanelment Check -- Verifies treating hospital is on CIL empanelled list (1,000+ hospitals)
- Interactive Dashboard -- Review, edit, and save extracted data via web UI
- 100% Local -- No cloud API calls; all AI runs locally via Ollama

Technology Stack

Layer	Technology	Purpose
Frontend	Vanilla HTML + JS + CSS	Dashboard, upload, SSE progress
Backend	Python 3.11 + FastAPI	REST API, async processing
OCR	Tesseract + pdf2image	PDF -> text extraction
AI/LLM	Ollama + Phi-3:3.8b	Structured data extraction
Database	SQLite with FTS5	CGHS rates, hospitals, bills
Container	Docker + start.sh	Deployment on Render/Koyeb

2. System Architecture

```
USER BROWSER (dashboard/index.html + app.js + styles.css)
  Tabs: Dashboard | Upload Bill | Rules Catalog
    |
    | HTTP REST + SSE (port 8000, same origin)
    v
FASTAPI SERVER (server.py)
  POST /api/upload      -> create job, start async task
  GET  /api/jobs/{id}/stream -> SSE real-time progress
  GET  /api/bills       -> all processed bills
  GET  /api/bills/{id}  -> full bill detail + validation
  POST /api/bills/{id}/save -> persist user edits
  GET  /api/cghs/search  -> CGHS procedure rate lookup
  GET  /api/hospitals/search -> hospital empanelment lookup
  GET  /api/status      -> Ollama AI health check
    |                    |
    v                    v
BillExtractionPipeline  ValidationEngine
(scripts/extract_bill.py) (scripts/validation_engine.py)
  PDF Repair (ghostscript)  Loads 39 rules from YAML
  OCR (Tesseract)           Queries SQLite for rates
  LLM parse (Ollama/Phi-3)  Returns ValidationResult list
  Returns BillData object
    |                    |
    ++++++
    |
    SQLite Database
    data/processed/medical_bills.db
    cghs_rates + FTS5 index (3,000+ procedures)
    hospitals  + FTS5 index (1,000+ hospitals)
    employees, medical_claims, claim_line_items
```

3. Processing Pipeline

Stage 1 -- PDF Repair:

Uses ghostscript (gs) to re-distill the uploaded PDF, fixing structural corruption. Saved as data/processed/bills/{id}_repaired.pdf. If GS not installed, original PDF is used.

Stage 2 -- OCR Extraction:

PDF converted to page images via pdf2image (poppler). Images preprocessed (grayscale, adaptive threshold, deskew) then fed to Tesseract. Per-page confidence averaged to produce OCR confidence score.

Stage 3 -- AI Structuring (Phi-3):

Raw OCR text sent to Phi-3:3.8b model via Ollama HTTP API. Prompt instructs model to extract structured JSON with patient, hospital, admission, financial, and line item fields. Falls back to regex extraction if Ollama offline.

Stage 4 -- Rule Validation:

Extracted BillData passed to ValidationEngine. Each of 39 rules evaluated, returning status (PASS/FAIL/WARN), human-readable message, and optional financial impact amount.

4. File-by-File Guide

```
PROJECT/
??? server.py          <- FastAPI backend (main entry point, 830 lines)
??? start.sh           <- Production startup script (Koyeb/Render)
??? Dockerfile         <- Docker container definition
??? render.yaml        <- Render.com deployment config
??? requirements.txt    <- Python pip dependencies
??? dashboard/
?   ??? index.html     <- Single-page app shell
?   ??? app.js         <- All frontend JS (1,240 lines)
?   ??? styles.css     <- Dashboard styles (1,040 lines)
??? scripts/
?   ??? extract_bill.py <- OCR + LLM extraction pipeline (1,251 lines)
?   ??? validation_engine.py <- 39-rule validation engine (640 lines)
?   ??? setup_database.py <- SQLite schema creation (494 lines)
?   ??? parse_cghs_rates.py <- CGHS rate card PDF parser (528 lines)
?   ??? parse_hospital_list.py <- Hospital empanelment list parser (369 lines)
?   ??? ocr_rules.py    <- OCR extraction from rule PDFs (227 lines)
??? data/
??? validation_rules.yaml <- 39 rules in structured YAML (685 lines)
??? raw/                <- Raw parsed text from PDFs
??? processed/
    ??? medical_bills.db <- Main SQLite database
    ??? cghs_rates.csv   <- CGHS procedures (3,000+ rows)
    ??? hospitals.csv    <- Empanelled hospitals (1,000+ rows)
    ??? bills/           <- Per-bill JSON files
```

server.py -- The Backend

- Startup: loads all previously-processed bill JSON files into memory (processed_bills list)
- Upload endpoint: saves PDF to data/uploads/, creates job dict, starts asyncio background task
- Async processing: runs 4-stage pipeline in thread pool executor (run_in_executor) to avoid blocking event loop
- SSE stream: /api/jobs/{id}/stream polls the job dict and sends updates every 500ms as text/event-stream
- In-memory storage: jobs and bills stored in Python dicts; bills reloaded from JSON on restart

extract_bill.py -- BillExtractionPipeline

- process_bill(pdf_path) -- main entry point: repair -> OCR -> LLM/regex -> return BillData
- _ocr_page(image) -- preprocesses image, runs Tesseract, computes confidence
- _select_key_pages(ocr_text) -- picks most informative pages to send to LLM (avoids token overflow)
- _call_ollama(prompt) -- sends prompt to Ollama HTTP API, retries on timeout
- _extract_with_regex(text) -- fallback extraction using 30+ regex patterns
- BillData, PatientInfo, HospitalInfo, AdmissionInfo -- structured dataclasses for extracted data

validation_engine.py -- ValidationEngine

- validate_claim(claim_dict) -- top-level; calls all 15 private check methods
- _parse_date(raw) -- multi-format parser: YYYY-MM-DD, DD/MM/YYYY, DD-MM-YYYY, DD.MM.YYYY
- find_cghs_rate(name) -- FTS5 full-text search + LIKE fallback in cghs_rates table
- verify_hospital(name) -- FTS5 + LIKE lookup in hospitals table
- SQLite conn.row_factory = sqlite3.Row -- enables dict-style column access

5. Validation Rules Engine -- All 39 Rules

Rule ID	Category	Description	Severity
E001	Eligibility	Patient must be employee or registered dependent	ERROR
E002	Eligibility	Working spouse -> use MAR scheme, not CPRMSNE directly	ERROR
E003	Eligibility	Hospital must be empanelled with CIL for treatment	ERROR
E004	Eligibility	Admission within 45 days of referral date	ERROR
R001	Rate	All charges <= CGHS rates (NABH/Non-NABH as applicable)	ERROR
R002	Rate	No CGHS rate -> use AIIMS rate; flag for manual review	WARN
R003	Rate	OPD consultation <= CGHS rate (Rs350 Non-NABH/Rs400 NABH)	ERROR
R004	Rate	Investigation amounts <= CGHS rates	ERROR
RR001	Room Rent	Room category must match grade entitlement (CIL OM Nov 202	ERROR
RR002	Room Rent	Room rent per day <= CGHS ceiling for entitled category	ERROR
RR003	Room Rent	Bed days = discharge - admission (no times); 24h periods i	WARN
P001	Package	No separate charges (room/consult/OT/medicines) during pac	ERROR
P002	Package	Package includes 2 pre-op + 2 post-op consultations	ERROR
P003	Package	Pre-admission fitness tests part of package for elective s	ERROR
P004	Package	Package starting 4+ days post-admission needs discharge su	WARN
P005	Package	Second surgery in same admission reimbursed at 50%	ERROR
C001	Consult	Consultations must match doctors in discharge summary	WARN
C002	Consult	First/last day = 1 consultation each when times are given	WARN
PH001	Pharmacy	Only reimbursable medicines (no cosmetics, toiletries, OTC	ERROR
PH002	Pharmacy	Medicine quantity on bill must match prescription	WARN
PH003	Pharmacy	Medicines during package period not separately billable	ERROR
D001	Docs	Test reports must be attached for major investigations	WARN
D002	Docs	Blood transfusion: discharge summary + documentary proof r	ERROR
D003	Docs	Implant bill: patient+hospital name + SECL doctor signatur	ERROR
D004	Docs	Each bill page must be signed by SECL scrutiny officer	WARN
D005	Docs	Discharge summary mandatory for all IPD claims	ERROR
HV001	High Value	>=Rs5L -> 2 medical officers; >=Rs10L -> 3 doctors require	ERROR
HV002	High Value	Stay >15 days requires CMS approval (from 15 Mar 2016)	ERROR
OPD001	OPD	CPRMSNE OPD limit: Rs25,000 per card per financial year	ERROR
OPD002	OPD	CPRMSE domiciliary claims deducted from Rs25L overall limi	WARN
SP001	Spectacles	Grade-wise ceiling per 2-year block (BOARD Rs50K -> NON_EX	ERROR
SH001	Special	Shankar Nethralaya, CMC Vellore, AIIMS -> negotiated rates	INFO
DN001	Deceased	6 documents required for reimbursement to undeclared nomin	WARN
F001	Fraud	No duplicate: same hospital + admission date + patient	ERROR
F002	Fraud	Flag charges >150% of CGHS rate or >2sigma above mean	WARN
F003	Fraud	Procedures must be clinically consistent with diagnosis	WARN
F004	Fraud	Flag >6 claims/year for same patient	WARN
DC001	Disease	Critical/Non-critical classification required for CPRMSE/N	WARN
SAP001	SAP	Disallowed + approved = claimed; sign before SAP entry	INFO

6. API Reference

Method	Endpoint	Description
POST	/api/upload	Upload PDF bill. Params: use_llm, dpi, max_pages. Re
GET	/api/jobs/{id}	Poll job status, progress, steps, and result.
GET	/api/jobs/{id}/stream	SSE stream -- real-time pipeline progress events.
GET	/api/bills	List all processed bills (summary).
GET	/api/bills/{id}	Full bill detail with line items and validation resu
POST	/api/bills/{id}/save	Save user-edited bill. Body: full bill JSON.
GET	/api/cghs/search?q=	Search CGHS rate card by procedure name (FTS5).
GET	/api/hospitals/search?q=	Search empanelled hospitals list.
GET	/api/status	System status: Ollama AI health, DB, bill count.

Bill JSON Structure (Key Fields)

```
{
  "id": "uuid-v4",
  "patient": { "name": "...", "employee_id": "...", "uhid": "...",
    "age": 45, "gender": "M", "relationship": "SELF" },
  "hospital": { "name": "...", "city": "...", "nabh_status": "NABH" },
  "admission": { "admission_date": "DD/MM/YYYY", "discharge_date": "...",
    "days_stayed": 5, "diagnosis": "...", "treating_doctor": "..." },
  "bill_number": "...", "bill_date": "...",
  "total_amount": 85000.0, "net_amount": 82000.0,
  "discount": 3000.0, "advance_paid": 20000.0, "balance_due": 62000.0,
  "line_items": [
    { "item_type": "CONSULTATION", "description": "...", "quantity": 1, "amount": 500.0 }
  ],
  "confidence_scores": { "patient_name": 0.92, "hospital_name": 0.88, ... },
  "validation_results": [
    { "rule_id": "E003", "status": "pass", "message": "Hospital verified in empanelled list" }
  ],
  "ocr_confidence": 78.4, "extraction_method": "OCR_LLM",
  "total_pages": 12, "extraction_timestamp": "2026-03-04T07:00:00"
}
```

7. Dashboard Guide

Dashboard Tab

- Stats row: Bills Processed, Rules Passed, Issues Flagged, Total Amount Claimed
- Bill Selector -- switch between processed bills (shown when >1 bill)
- 4 Info Cards: Patient Details, Hospital, Admission Dates, Financial Summary
- Field Confidence Panel -- colour-coded bars (red = below 80% confidence threshold)
- Review & Edit panel -- slide-in form to correct any OCR extraction errors
- Line Items table -- all extracted charges with type, description, quantity, amount
- Validation Results grid -- pass/fail/warn for each rule with message
- Extraction Metadata -- method used, OCR %, total pages, timestamp, source file

Upload Tab

- Drag-and-drop or click-to-browse PDF upload (accepts by MIME type or .pdf extension)
- Options: Enable/disable AI (Phi-3) | OCR DPI (150/200/300) | Max pages limit
- Live pipeline: 4 steps each with animated progress bar, updated via SSE

- On completion, auto-redirects to Dashboard tab with the new bill selected

8. All 23 Bugs Found & Fixed

All 23 bugs have been fixed and pushed to GitHub in two commits: a814c97 and 9831720.

ID	File	Severity	Description	Fix
B01	server.py	Critical	save_bill bare dict type hint -> 422 error o	Added Body(...) annotati
B02	server.py	Critical	Shared dict between processed_bills and job	Used dict() copy for bot
B03	server.py	Critical	NameError: variable 'i' undefined when proce	Introduced save_index va
B04	server.py	Critical	Blocking requests.get() in async function ->	Wrapped in run_in_execut
B05	server.py	Critical	Deprecated asyncio.get_event_loop() -> error	Changed to get_running_l
B06	server+engine	Critical	Date format mismatch: engine expected YYYY-M	Added _normalise_date()
B07	server.py	Critical	Key mismatch: 'total_claimed' vs 'claimed_am	Added both keys to claim
B08	server.py	Major	Port mismatch: __main__ said 8080, start.sh	Unified to 8000 everywhe
B09	server.py	Major	/api/hospitals/search queried non-existent '	Removed state from query
B10	extract_bill.py	Major	City regex had no capture group -> IndexErro	Wrapped in capturing gro
B11	extract_bill.py	Moderate	OCR confidence inflated: 0% confidence words	Changed >0 to >=0 (only
B12	extract_bill.py	Moderate	Unhandled ValueError in _select_key_pages pa	Added try/except (ValueE
B13	validation_engi	Critical	_check_referral_validity hard-coded %Y-%m-%d	Uses new _parse_date() h
B14	validation_engi	Critical	_check_bed_days same date format issue	Uses _parse_date()
B15	validation_engi	Critical	_check_extended_stay same date format issue	Uses _parse_date()
B16	ocr_rules.py	Moderate	Unhandled FileNotFoundError when pdftotext n	Added except FileNotFoun
B17	setup_database.	Moderate	Single commit at end -- any failure rolls ba	Split into 3 independent
B18	parse_cghs_rate	Moderate	ZeroDivisionError when table is empty (100*n	Changed to max(total, 1)
B19	app.js	Major	Drag-drop rejected PDFs with empty MIME type	Also checks file.name.en
B20	app.js	Moderate	Concurrent renderDashboard() calls possible	Added state._rendering b
B21	render.yaml	Major	PORT=8080 in Render config but start.sh and	Fixed to 8000
B22	requirements.tx	Moderate	Unused aiofiles; missing system deps documen	Removed unused dep; adde
B23	validation_engi	Moderate	OPD limit rule checks 'CPRMSNE' but server a	Noted for future employe

9. Setup & Running Locally

PREREQUISITES: Install system tools BEFORE pip install (not available via pip):

macOS: `brew install tesseract poppler ghostscript`

Linux: `sudo apt-get install tesseract-ocr poppler-utils ghostscript`

```
# 1. Clone repository
git clone https://github.com/kumarnitn/MEDICAL-BILL-SCRUTINY-SYSTEM.git
cd MEDICAL-BILL-SCRUTINY-SYSTEM

# 2. Create and activate virtual environment
python3 -m venv .venv && source .venv/bin/activate

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. Set up the SQLite database
python scripts/setup_database.py

# 5. Parse CGHS rate card (requires raw text extracted from CGHS Rate.pdf)
python scripts/parse_cghs_rates.py

# 6. Parse hospital empanelment list
python scripts/parse_hospital_list.py

# 7. (Optional) Install Ollama for AI extraction
#   Download from https://ollama.ai, then:
ollama pull phi3:3.8b

# 8. Start the server
python server.py
#   -- or via uvicorn --
uvicorn server:app --host 0.0.0.0 --port 8000 --reload

# Open browser at: http://localhost:8000
```

TIP: Without Ollama, the system automatically uses regex-based extraction. Accuracy is lower but all features work. You can enable Ollama later and re-process bills.

10. Deployment

Docker (Recommended)

```
docker build -t medbill-ai .
# Mount data/ as volume to persist DB and bills between restarts
docker run -p 8000:8000 -v $(pwd)/data:/app/data medbill-ai
```

The Dockerfile automatically installs tesseract, poppler, and ghostscript. No manual system dep setup needed in Docker.

Render.com / Koyeb (Cloud)

- render.yaml and start.sh are pre-configured for one-click cloud deployment
- Service type is 'docker' -- uses the Dockerfile directly
- PORT is set to 8000 via environment variable (fixed from 8080 bug)
- NOTE: Ollama (local LLM) is NOT available in cloud deployments -- system runs in regex-fallback mode
- SQLite database is ephemeral in cloud -- mount a persistent volume or use cloud storage for production

Environment Variables

Variable	Default	Description
PORT	8000	Port the server listens on
OLLAMA_BASE_URL	http://localhost	Ollama API endpoint
DB_PATH	data/processed/m	SQLite database path